

# Table des matières

|                                                             |    |
|-------------------------------------------------------------|----|
| Énoncé du sujet .....                                       | V  |
| Introduction .....                                          | 1  |
| 1. Présentation du système .....                            | 5  |
| 1.1. Principes .....                                        | 5  |
| 1.2. Ressources .....                                       | 6  |
| 1.3. Administration du système .....                        | 7  |
| 1.4. Cycle de vie du système .....                          | 8  |
| 1.5. Exemple d'utilisation .....                            | 9  |
| 2. Conception du système .....                              | 10 |
| 2.1. Modèle de ressource .....                              | 10 |
| 2.1.1. Structure d'une ressource .....                      | 10 |
| 2.1.2. Statut d'une ressource .....                         | 10 |
| 2.1.3. Relations entre les ressources .....                 | 12 |
| 2.1.4. Ressources partagées et ressources mutualisées ..... | 12 |
| 2.1.5. Catégories de ressources .....                       | 16 |
| 2.1.6. Synthèse .....                                       | 18 |
| 2.2. Orchestrateur et réconciliation .....                  | 19 |
| 2.2.1. Orchestrateur .....                                  | 19 |
| 2.2.2. Contrôleur et déclarativité .....                    | 19 |
| 2.2.3. Orchestration de la réconciliation .....             | 21 |
| 2.2.4. Boucle de réconciliation .....                       | 25 |
| 2.2.5. Suppression des ressource .....                      | 25 |
| 2.2.6. Réaction aux événements internes .....               | 26 |
| 2.2.7. Réaction aux événements externes .....               | 26 |
| 2.3. Initialisation du système .....                        | 27 |
| 2.4. Démarrage des processus .....                          | 27 |
| 3. Implémentation .....                                     | 29 |
| 4. Tests et validation .....                                | 30 |
| 5. Comparaison avec les solutions existantes .....          | 31 |
| 6. Résultats .....                                          | 32 |
| 7. Discussion .....                                         | 33 |
| Conclusion .....                                            | 34 |

## Table des illustrations

|           |                                                                            |    |
|-----------|----------------------------------------------------------------------------|----|
| Figure 1  | Transition entre les différents status d'une ressource .....               | 11 |
| Figure 2  | Problème des ressources partagées implicitement .....                      | 13 |
| Figure 3  | Solution au problème du partage de ressources implicite .....              | 15 |
| Figure 4  | Création de sous-ressources à partir d'un parent .....                     | 18 |
| Figure 5  | Synthèse des types de ressources et des liens .....                        | 19 |
| Figure 6  | Réconciliation d'une ressource .....                                       | 20 |
| Figure 7  | Orchestraion de la réconciliation dans un modèle centralisé .....          | 21 |
| Figure 8  | Orchestraion de la réconciliation dans un modèle décentralisé .....        | 22 |
| Figure 9  | Démarrage du système .....                                                 | 27 |
| Figure 10 | Arbre d'exécution des processus du système .....                           | 28 |
| Tableau 1 | Types de ressources et leur propriétés .....                               | 17 |
| Tableau 2 | Comparaison des modèles d'orchestration centralisés et décentralisés ..... | 24 |

**TODO:**

- Acronyms list
- Table of appendices

**TODO:** Remerciements

**OS POUR LE DÉPLOIEMENT DE SERVICES CONTENEURISÉS****ORIENTATION: INFORMATIQUE LOGICIELLE****Descriptif:**

Le déploiement d'applications conteneurisées s'appuie désormais largement sur des approches déclaratives, en particulier dans les environnements clusterisés. En revanche, la configuration du système hôte reste souvent prise en charge par des outils impératifs distincts, ce qui introduit une rupture entre la gestion de l'infrastructure sous-jacente et celle des applications.

Cette séparation limite l'automatisation complète du déploiement et complique la reproductibilité, en particulier dans les contextes de déploiement unitaire, où les solutions conçues pour des environnements clusterisés peuvent s'avérer peu adaptées.

Le but de ce projet est donc de pallier à ce manque, en créant une distribution Linux légère destinée à la mise en place d'infrastructures conteneurisées. Ce système d'exploitation devra être léger en ressources, configurable via une API entièrement déclarative et sécurisé (conteneurs rootless, etc.).

**Travail demandé:**

- Système développé ne se basant pas sur une distribution Linux existante.
- Gestion de la configuration du système réalisée de manière déclarative.
- Gestion des ressources nécessaires à la conteneurisation (conteneurs, interfaces réseau Ethernet) via ces fichiers déclaratifs.
- Exécution des conteneurs de manière rootless afin d'augmenter la sécurité.
- Administration du système uniquement via une API, sans accès distant (SSH) ou interface procédurale.
- Test et validation de la solution réalisée sur des use-cases réels.
- Comparaison de la solution développée à Talos Linux et NixOS.
- Création d'un système permettant de facilement déployer/installer la solution sur machine physique ou infrastructure Cloud.

Candidat:  
**ARROYO FRÉDÉRIC**

Filière d'études: ISC

Professeur responsable:  
**GLÜCK FLORENT**

**En collaboration avec:** n/a

Travail de bachelor soumis à une  
convention de stage en entreprise: non

Travail soumis à un contrat de  
confidentialité: non

**TODO:** Résumé

# Introduction

## Contexte et problématique

**TODO:** Introduction / contexte et problématique

- Ajouter les références aux divers outils mentionnés.

La conteneurisation s'est imposée comme le mode standard de déploiement des applications dans les environnements modernes. Les outils qui gravitent autour d'elle, tels que Docker Compose ou Kubernetes, adoptent un mode d'opération déclaratif dans lequel l'utilisateur décrit l'état désiré et le système se charge de le maintenir. Ce paradigme a également gagné l'infrastructure cloud avec des outils tels que Terraform. Il a donné naissance au modèle GitOps, qui consiste à stocker l'ensemble de la configuration dans Git. Ainsi, chaque fois qu'une mise à jour est effectuée, elle est appliquée automatiquement, ce qui rend le déploiement et l'infrastructure entièrement automatisés. Dans ces approches, l'interaction directe avec le système d'exploitation devient l'exception plutôt que la règle.

Toutefois, les systèmes d'exploitation sous-jacents reposent encore majoritairement sur des distributions génériques, administrées avec des outils de gestion de configuration dont le modèle d'exécution reste fondamentalement impératif, ce qui complique l'automatisation et la reproductibilité. De plus, ces distributions n'intègrent pas la conteneurisation comme un élément de première classe du système et nécessitent de configurer et gérer les conteneurs de manière distincte du système d'exploitation. De ce fait, la complexité de la configuration et du maintien en état repose entièrement sur l'administrateur.

Certaines solutions plus spécialisées existent, alliant déclarativité et support natif pour la conteneurisation, mais elles souffrent de lacunes diverses. NixOS permet une gestion déclarative de l'ensemble du système, hôte et conteneurs, mais cette déclarativité est ponctuelle: l'état désiré n'est appliqué qu'à un moment donné, et aucune boucle de contrôle ne corrige les dérives ultérieures. Flatcar Container Linux offre une base minimale et orientée conteneurs, mais ne rend déclaratif que la phase d'installation initiale de l'hôte. D'autres solutions, telles que Talos Linux, permettent une administration entièrement déclarative et continue, mais s'intègrent étroitement avec Kubernetes, au prix d'une complexité opérationnelle et d'une empreinte mémoire disproportionnées pour le simple déploiement de quelques conteneurs.

Il manque donc un système d'exploitation capable de décrire la configuration de l'hôte et des conteneurs dans un modèle déclaratif unique et homogène, et de maintenir cet état de manière continue et autonome. Un tel système serait particulièrement pertinent pour les déploiements modestes, où la complexité d'un orchestrateur complet n'est pas justifiée, mais

où l'on souhaite néanmoins disposer d'un système déclaratif capable de se maintenir en état de manière autonome.

## Objectifs

Le travail de semestre effectué préalablement a permis d'analyser en profondeur les forces et les faiblesses des différentes solutions disponibles. Il a débouché sur la présentation d'une architecture de très haut niveau, explorant les concepts fondamentaux et les briques techniques nécessaires à l'implémentation d'une solution adaptée. Le présent travail reprend ces conclusions afin de concevoir une architecture détaillée, puis de l'implémenter, de la valider et de la comparer à Talos Linux et NixOS.

L'objectif central est de fournir un système d'exploitation entièrement configurable selon un modèle déclaratif unique, dans lequel la configuration de l'hôte et celle des conteneurs sont intégrées de manière homogène. La conteneurisation est traitée comme un élément natif du système: les conteneurs sont décrits et gérés au même titre que les autres ressources, sans couche externe. En particulier, le système doit maintenir l'état déclaré de manière continue et autonome, en s'appuyant sur une boucle de contrôle qui surveille en permanence l'état réel, détecte automatiquement les dérives et les corrige sans intervention humaine.

Parce que le système est destiné à des déploiements exposés à Internet et fonctionne sans surveillance constante, la sécurité est une exigence primordiale. Elle se concrétise par une isolation forte des composants système entre eux, une réduction maximale de la surface d'attaque et l'absence de tout accès interactif direct (notamment pas d'accès SSH). Le mode d'interaction entièrement déclaratif et automatisé permet de rendre **<TODO> Trouver un mot qui va bien </TODO>** viable.

Pour garantir une empreinte mémoire minimale et une maîtrise complète sur le fonctionnement du système, la solution est construite directement au-dessus du noyau Linux, sans recourir à une distribution préexistante. Elle comprend un processus d'initialisation (PID 1), un environnement utilisateur (user-space) restreint (sans shell ou systemd) qui gère le système, et intègre uniquement le runtime de conteneurs et un bootloader comme composants externes<sup>1</sup>. Ce choix de conception permet de réduire la complexité superflue et de limiter encore la surface d'attaque.

**<TODO> compléter la liste des « composants externes » si on en inclut d'autres </TODO>**

---

<sup>1</sup>Par « composant externe », il est entendu ici un logiciel tiers fonctionnant comme un service ou un processus indépendant, par opposition aux bibliothèques logicielles (souvent appelés packages ou librairies).

## Cadre du travail et méthodologie

Ce travail s'inscrit dans l'obtention du titre de Bachelor of Science en Informatique et systèmes de communication, orientation Informatique logicielle, à la Haute école du paysage, d'ingénierie et d'architecture de Genève (HEPIA). Réalisé à plein temps du 11 mai au 19 août 2026, il reflète l'état des connaissances et des technologies à cette période. Il a été précédé d'un travail de semestre qui a permis une analyse détaillée de la problématique et l'établissement des premiers éléments conceptuels et technique. L'ensemble du code source produit est disponible sur le dépôt Git institutionnel à l'adresse suivante: [https://gitedu.hesge.ch/flg\\_bachelors/tb/2026/container-infrastructure-deployment-os](https://gitedu.hesge.ch/flg_bachelors/tb/2026/container-infrastructure-deployment-os).

Des intelligences artificielles (IA) génératives ont été utilisées de manière ponctuelle pour améliorer la qualité rédactionnelle; un texte de base contenant l'intégralité du fond et de l'organisation a toujours été fourni en amont. Pour le code, elles ont principalement servi d'outil d'analyse qualitative, en complément des outils d'analyse de code traditionnels, afin de détecter d'éventuelles failles, bogues ou usages contraires aux bonnes pratiques. L'IA a également été employée pour conforter l'exhaustivité des recherches documentaires et des efforts de débogage: une fois la collecte humaine jugée suffisante ou qu'elle se heurtait à une impasse, elle a été interrogée pour signaler d'éventuels angles morts, en suggérant des mots-clés ou des références complémentaires. Ces suggestions ont toujours été vérifiées avant d'être utilisées. L'IA n'a explicitement pas été utilisée dans le cadre de la production de code, pour prendre décisions architecturales ou organisationnelles, ainsi que pour effectuer recherches et débogage initiaux. Un compte rendu détaillé et exhaustif de ces usages figure en annexe <TODO> Référencer l'annexe. </TODO>.

Le travail a été structuré en trois jalons principaux. Une première phase conceptuelle, jusqu'à la fin mai, a permis de poser les bases technologiques. Une phase de développement a suivi jusqu'à fin juin, intégrant l'ensemble des fonctionnalités strictement nécessaires à l'obtention d'une solution testable et comparable répondant aux objectifs techniques de l'énoncé. Enfin, l'implémentation de fonctionnalités additionnelles et l'amélioration continue de la solution ont occupé la période allant jusqu'à fin juillet.

## Structure du document

Le reste du document est organisé en six chapitres. Le premier chapitre présente le système du point de vue de l'utilisateur: il décrit le mode d'interaction déclaratif, le cycle de vie des conteneurs, le fonctionnement attendu de la boucle de contrôle et les principes de configuration. Les notions nécessaires à la compréhension de la solution y sont introduites, sans entrer dans les détails internes. Le deuxième chapitre expose la conception détaillée. Il définit l'architecture

générale, l'organisation des composants, leurs responsabilités et leurs interfaces. Les structures de données propres au modèle déclaratif et au mécanisme de réconciliation y sont spécifiées. Le troisième chapitre traite de l'implémentation. Il détaille les choix techniques (langage, bibliothèques, structures de code), les algorithmes principaux et la manière dont la boucle de contrôle a été réalisée. L'environnement de développement et les outils de test sont également présentés dans ce chapitre. Le quatrième chapitre est consacré à la validation. Il expose la méthodologie employée pour tester le système (tests unitaires, tests d'intégration, scénarios de dérive) et synthétise les résultats obtenus, notamment en matière de réactivité et de fiabilité du maintien d'état. Le cinquième chapitre compare la solution à Talos Linux et NixOS. La méthodologie de comparaison y est précisée, le protocole de mesure et analyse des résultats. Enfin, le sixième chapitre dresse un bilan global, discute les forces et les limites de l'approche, et esquisse des perspectives d'amélioration et de travaux futurs.

# 1. Présentation du système

## Note:

- Présente le système pour un utilisateur.
- Mélange un petit peu un « Quick Start » / « Getting Started » et un « Learn More » de haut niveau
- Volume estimé: 10-13 vrai pages

<TODO> Amorce du chapitre + plan interne + synthèse brève. </TODO>

<TODO> Référencer les divers outils. </TODO>

## 1.1. Principes

Le système s'articule autour de cinq principes fondamentaux: la déclarativité, l'automatisation, la légèreté, la sécurité par défaut et la portabilité. Ces principes ont été dégagés de l'analyse des besoins présentés dans le projet de semestre, et répondent à un contexte de déploiement particulier: des environnements hétérogènes, non distribués, parfois sur du matériel aux ressources contraintes et administrés par un utilisateur seul.

### 1.1.1. Déclarativité

Tout d'abord, plutôt que de décrire les étapes à exécuter pour atteindre un état donné, l'administrateur décrit directement l'état final souhaité. Le système détermine lui-même les actions nécessaires pour y parvenir, quelle que soit l'état courant de la machine.

Ce choix est particulièrement adapté à un contexte de reconfiguration fréquente sur des environnements variés. Une approche impérative obligerait l'administrateur à prendre en compte l'état courant avant chaque opération et rend plus difficile la récupération en cas d'erreur durant la configuration du système. La déclarativité permet d'améliorer la reproductibilité des déploiements et simplifie le retour à un état antérieur en cas d'erreur humaine.

### 1.1.2. Automatisation

Toute opération administrative, depuis l'installation initiale jusqu'à la reconfiguration, doit pouvoir être réalisée sans intervention manuelle. L'objectif recherché est de permettre une intégration simple dans le processus de déploiement, en particulier lorsque celui-ci repose sur une approche GitOps/DevOps.

Les interventions manuelles pour le diagnostic ou la récupération restent possibles, mais doivent demeurer exceptionnelles et clairement distinctes du mode normal d'exploitation.

### **1.1.3. Légèreté**

Le système doit présenter une empreinte mémoire minimale afin de laisser un maximum de ressources aux services déployés par l'administrateur. De fait, il n'assume qu'un seul rôle: servir de plateforme d'exécution pour des services conteneurisés et aucun composant superflu n'est présent.

Ce principe se justifie par des déploiement sur des appareils basse consommation disposant de peu de ressources. Cela présente en outre deux effets secondaires bénéfiques: une configuration plus simple en raison du petit nombre de composants et une surface d'attaque plus réduite.

### **1.1.4. Sécurité par défaut**

Le système adopte des paramètres restrictifs dès l'installation, sans configuration supplémentaire. Les services internes et ceux déployés par l'administrateur sont isolés aussi fortement que possible, afin de limiter les risques de mouvement latéral en cas de compromission d'un composant.

L'analyse des besoins révèle principalement un usage pour l'hébergement de services accessible via Internet et donc soumis en permanence à des tentatives d'intrusion. Des mécanismes d'exception strictement contrôlés restent disponibles pour les cas qui le requièrent, notamment lorsque l'hôte assume un rôle de routeur ou de pare-feu réseau.

### **1.1.5. Portabilité**

L'interface d'administration et le format de configuration doivent rester identiques quelle que soit la configuration matérielle ou réseau sous-jacente. En dehors des dépendances introduites explicitement par l'administrateur, le transfert du système d'un environnement à un autre ne doit nécessiter aucune adaptation de la configuration existante.

Dans un contexte où un même administrateur administre plusieurs machines aux profils distincts, cette homogénéité réduit la charge cognitive et rend les procédures opérationnelles transférables sans friction.

## **1.2. Ressources**

### **1.2.1. Définition**

L'administration et la configuration du système est organisé autour des ressources. Une ressource est simplement l'abstraction d'un ou plusieurs objets concrets au sein du système. Par exemple, la ressource « conteneur » représente le concept du même nom; la ressource « interface réseau » représente quant à elle un lien réseau, une ou plusieurs routes, et une ou plusieurs adresses.

### 1.2.2. Réseau

**TODO:** Firewall, VPN, DHCP, IP statique, IPv6, etc.

### 1.2.3. Conteneurisation

**TODO:** Conteneur, gestion des images, registres privé, Podman, Secrets, Limites

### 1.2.4. Stockage et volumes

**TODO:** Partitionnement disque, volume, cache, data, chiffrement (on peut combiner plusieurs méthodes de chiffrement!)

### 1.2.5. Portabilité de la configuration

**TODO:** Portabilité

- On peut matcher une interface par une MAC address (partielle), nom, index, etc.
- Idem pour les disques
- En tous les cas, si une config a trois interface réseau mais que physiquement il y en a que une sur la nouvelle machine, pas de magie

## 1.3. Administration du système

### 1.3.1. Exigences matérielles

**TODO:** Hardware requirements

- RAM
- Storage

### 1.3.2. Interface d'administration

**TODO:** Interface d'administration

- Parler de la CLI avec un tableau des commandes les plus importantes
- Parler de Terraform vite fait
- Parler de comment on s'authentifie à l'interface (token, mTLS, etc.)
- Parler des commandes importantes
  - Push config
  - Voir l'état
  - Téléverser des fichiers
  - Diagnostic réseau
  - exec dans un conteneurs
  - les logs
  - reboot

### 1.3.3. Sources de configuration

**TODO:** Config src

- disk
- remote
- cloud-init
- etc.
- Schéma des sources

### 1.3.4. Monitoring et observabilité

**TODO:** Monitoring

- En gros logs accessible via le CLI, ou exporté via les trucs standard (Loki, ES, etc.)
- Pour l'observabilité, idem + exporté via Prometheus
- Export des métriques et logs host **ET** conteneurs

### 1.3.5. Persistence des données

**TODO:** Persistence des données

- Comment consulter le stockage
- Parler du fait que le disque est auto-déverrouillé + risques que cela implique
- Parler en particulier du risque du mode « passphrase » pour le chiffrement
- On peut aussi utiliser une passphrase en mode manuel, dans ce cas le système n'est pas auto-déverrouillé

## 1.4. Cycle de vie du système

### 1.4.1. Personnalisation de l'image

**TODO:** Personnalisation

- Comment intégrer de nouvelles options du noyau
- De nouveau drivers
- Pre-bake une image conteneur ou une config
- etc

### 1.4.2. Méthodes de démarrage

**TODO:** Boot

- boot en réseau
- mode in-memory
- différent bootloader

### 1.4.3. Installation initiale

**TODO:** Parler du fait qu'il suffit simplement de push une configuration valide sur un système vierge en maintenance.

### 1.4.4. Mise à jour et retours en arrière

**TODO:**

- Parler du mécanisme de mise à jour du système avec le champ `osVersion` et l'installation A/B.
- Parler du fait qu'on peut rollback la config ou la version du système (ou les deux).

### 1.4.5. Maintenance

**TODO:** Parler du mode de maintenance et du mode d'authentification additionnel propre à celui-ci.

### 1.4.6. Sauvegarde et restauration

**TODO:**

- Parler de comment backup/restore le système
- Comment exporter un volume/conteneur individuellement aussi

## 1.5. Exemple d'utilisation

**TODO:** Deployer un conteneur simple avec un serveur HTTP

## 2. Conception du système

### **TODO:** Conception du système

- Transition avec le chapitre précédant/amorce
- Objectifs du chapitre
- Plan interne

### **Missing:** Manques du chapitre conception du système tel que rédigé

- Génération du nom: le contrôleur se débrouille pour faire en sorte que les sous-ressources qu'il crée aient un nom unique
- Parler d'idempotence
- Parler de sécurité? Ou bien vu que c'est toujours des techno spécifique, à mettre dans l'implémentation?

### **Question:** Questions ouvertes

- Que se passe-t-il si par exemple on a une ressource `TlsPrivateKey` et qu'on dépend dessus? Peut-on lire la clef privée?
- De manière générale, comment faire remonter l'erreur lorsque, durant la réconciliation, le contrôleur retourne une réponse qui n'est pas valide (par exemple des dépendances qui ne sont pas réalisables?)

## 2.1. Modèle de ressource

### 2.1.1. Structure d'une ressource

Toute ressource partage une structure commune composée d'un type, d'un nom, d'une spécification, d'un état observé, d'un status et de métadonnées. Le type fixe le schéma structurel de la spécification et de l'état observé tandis que le nom identifie de manière unique une instance au sein de ce type. La spécification est l'état désiré, tandis que l'état observé reflète la réalité constatée par le contrôleur. L'état observé ainsi que les dépendances et autre sous-ressources sont établis à chaque réconciliation, tel qu'expliqué dans le [chapitre 2.2.2, Contrôleur et déclarativité](#). Par ailleurs, le type et le nom d'une ressource sont immuable; modifier l'un ou l'autre de ces champs est l'équivalent de supprimer la ressource avant les anciennes valeurs et en recrée une nouvelle. Entre deux réconciliation, l'état observé est celui résultant de la dernière réconciliation.

### 2.1.2. Statut d'une ressource

Le statut d'une ressource se divise en deux catégories: les statuts contrôlés par le contrôleur et ceux contrôlés par l'orchestrateur. Les statuts possibles sont:

- inconnu (*unknown*): état initial avant la première réconciliation ou lors du démarrage
- pas prêt (*not ready*): le contrôleur indique que la ressource n'est pas encore opérationnelle.
- prêt (*ready*): la ressource est conforme à sa spécification et pleinement fonctionnelle.
- erreur (*error*): une anomalie empêche d'atteindre l'état désiré; des détails (code, message) peuvent accompagner ce statut.
- terminé (*done*): la ressource a atteint un état final et ne nécessite plus de réconciliation. Si sa spécification est modifiée, l'orchestrateur la remplace automatiquement dans le statut inconnu avant de lancer une nouvelle réconciliation.
- en cours de suppression (*deleting*): la suppression est démarrée et est irréversible.

Lors de la réconciliation, le contrôleur retourne obligatoirement l'un des quatre statuts pas prêt, prêt, erreur ou terminé; les autres statuts sont gérés exclusivement par l'orchestrateur. Dans le cas des ressources statiques, le statut « en cours de suppression » est déclenché par le retrait de la ressource de la configuration. L'état d'erreur n'a pas de sémantique particulière lors de la planification de la réconciliation.

La status en *cours de suppression* est le seul status contrôlé par l'orchestrateur à être transitif au sein des ressources détenues. En ce qui concerne les status contrôlés par le contrôleur, ce dernier est libre de les rendre transitif ou non, mais cela se fait manuellement lors de la réconciliation. La machine d'états de transition des statuts est présentée dans la figure 1.

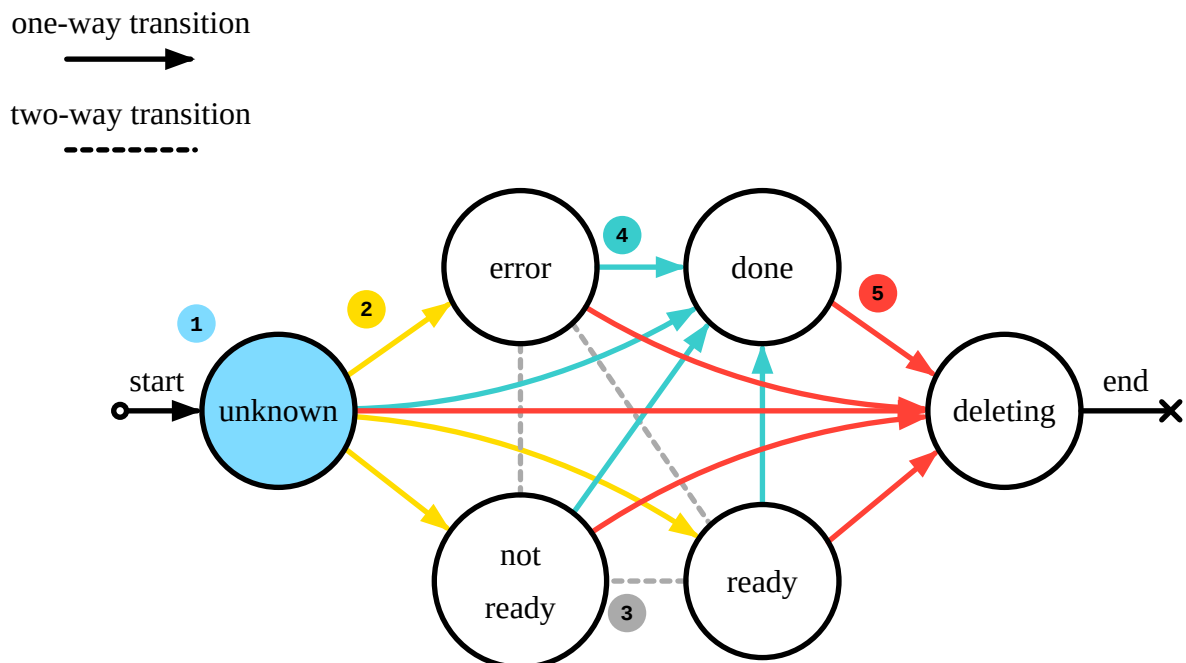


Figure 1. – Transition entre les différents status d'une ressource  
Illustre les divers status qu'une ressource peut avoir et quelles sont les transitions autorisées entre deux initialisations du système.

Tout ressource commence toujours dans le status inconnu **1**, de là, lors de la réconciliation la ressource basculera dans le status retourné par le contrôleur **2**. Cette transition ne peut s'effectuer qu'une seule fois par ressource et par démarrage. En effet, depuis ces status les ressources ne peuvent retourner dans le status inconnu que lors du redémarrage. En outre, les ressources peuvent librement passer à l'un ou l'autre des status déterminé par le contrôleur, sans règles particulières au niveau de l'orchestrateur **3**. Si le contrôleur place la ressource dans l'état terminé **4**, cette transition n'est réversible que dans le cas où la spécification de la ressource est mise à jour. Enfin, depuis n'importe quel état, il est possible de se rendre vers l'état en cours de suppression **5**, qui est définitivement irréversible.

### 2.1.3. Relations entre les ressources

Deux types de relations lient les ressources entre elles: la possession et la dépendance. Une ressource peut avoir au plus un détenteur; celui-ci dispose des droits de modification et de suppression sur la ressource, ainsi que le droit de consulter la spécification et l'état observé de celle-ci. La dépendance permet de consulter la spécification et l'état observé, sans pour autant pouvoir modifier la ressource. Le but de ce lien est de permettre de partager l'état observé d'une ressource à une autre ressource ne se situant pas dans l'arbre de possession. Dans le cas où la ressource se trouve au sein de l'arbre de possession, il est envisageable que les informations requises soit remontées au premier parent commun puis retransmises. Toutefois, toutes les ressources ne se situent pas dans un seul et même arbre de dépendance, comme abordé au sein du [chapitre 2.1.5, Catégories de ressources](#).

Ces liens ont divers impacts en ce qui concerne la planification de la réconciliation et la suppression des ressources, qui seront abordés respectivement dans les chapitres [2.2.4, Boucle de réconciliation](#) et [2.2.5, Suppression des ressources](#). Lors de l'ajout d'un tel lien, si l'une ou l'autre des ressources n'existe pas **<TODO> commenter remonter l'erreur? </TODO>**. Similairement, l'ajout d'un lien qui créerait un cycle est aussi empêché. En outre, ces relations ne sont jamais transitives: si *A* possède *B*, et *B* possède *C*, *A* ne possède pas *C*. Outre cela, il n'y a pas de limites particulières sur le nombre de dépendance entrante ou sortante.

### 2.1.4. Ressources partagées et ressources mutualisées

Le partage d'une ressource est naturellement réalisé par un lien de dépendance: plusieurs ressources peuvent consulter une même cible sans pouvoir la modifier. Dans la plupart des cas, il est possible de déterminer un seul et unique détenteur d'une telle ressource partagée. Toutefois, il existe certains cas où la ressource partagée existe de manière implicite au sein du système, et à travers différents arbres de possession, comme illustré dans la figure 2.

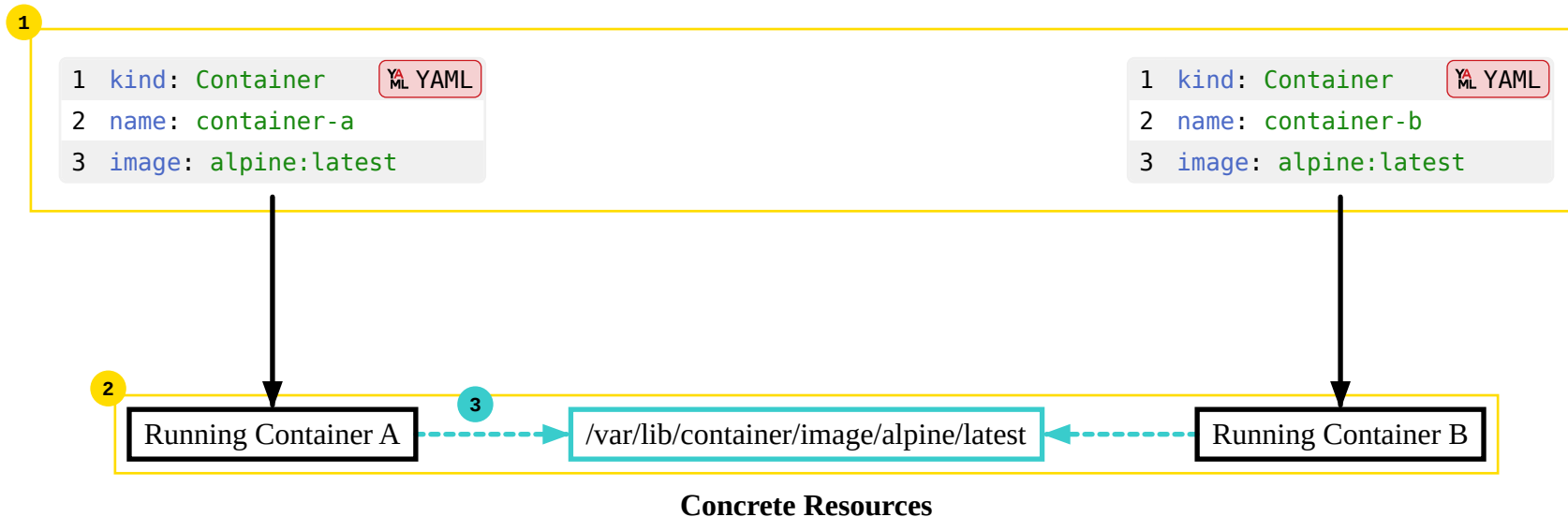


Figure 2. – Problème des ressources partagées implicitement

Illustre comment une ressource système va interagir avec une ressource concrète, et comment deux ressources concrètes différentes peuvent finalement interagir avec une seule et même autre ressource concrète.

Note: la configuration est abrégée à des fins d'illustration

Par exemple, deux configuration de conteneurs **1**, sans parent communs, crée deux conteneurs qui semble être indépendant **2**, mais ces deux conteneurs pointe en réalité vers une même ressource, en l'espèce, l'image **3**. Le système étant déclaratif, il faut disposer d'un moyen de supprimer cette image de manière déclarative.

Si l'utilisateur supprime l'une des deux configurations, la boucle de réconciliation associée tentera de détruire la ressource partagée, alors que celle-ci est encore requise par l'autre configuration. Selon la nature de la ressource sous-jacente, deux comportements sont possibles: soit la suppression échoue et la réconciliation reste bloquée, soit la ressource est détruite et doit être recréée par l'autre configuration.

Pour résoudre ce problème, le modèle introduit la notion de ressource mutualisée. Une ressource mutualisée est identifiée de manière unique par son type et son nom; elle ne possède aucune spécification. Contrairement aux ressources dynamiques ordinaires, elle n'appartient à aucun possesseur unique: l'orchestrateur en est le détenteur et, en raison de l'absence de paramètre, ne gère que la suppression de la ressource. En raison de l'absence de spécification, sa création est implicite: il suffit à une ressource de déclarer une dépendance vers une autre ressource mutualisée et l'orchestrateur se chargera de créer cette ressource. Si, ultérieurement, une autre ressource déclare dépendre de cette même ressource mutualisée, il n'y a pas de deuxième instance créée, elle va simplement référencer l'instance existante.

L'orchestrateur décidant de la suppression d'une ressource mutualisée, la règle est simple: dès lors qu'aucune autre ressource ne dépend sur une ressource mutualisée, alors celle-ci est placée en cours de suppression, et va suivre le même principe que les autres ressources, tel que détaillé au sein [chapitre 2.2.5, Suppression des ressources](#). De même, les mêmes principes de déclarativité, expliqués au sein du [chapitre 2.2.2, Contrôleur et déclarativité](#) sont appliqués à cette ressource, en particulier, même en l'absence d'une spécification propre, une ressource mutualisée peut créer des sous-ressources avec des configurations arbitraires.

La figure 3 illustre la résolution du conflit précédent grâce à une ressource mutualisée.

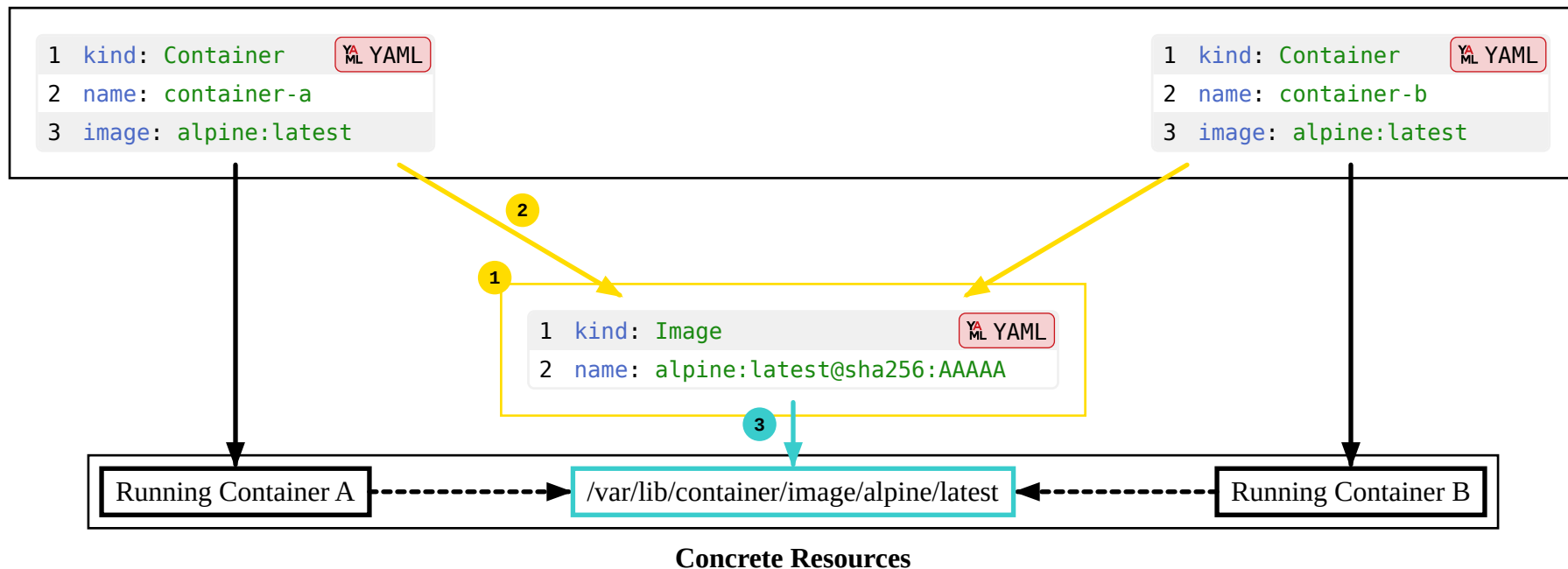


Figure 3. – Solution au problème du partage de ressources implicite

Illustre comment, en ajoutant une abstraction, la problématique de deux ressources concrètes indépendante, interagissant avec une troisième ressource concrète peut être résolue.

Note: la configuration est abrégée à des fins d'illustration

Dans la figure 3, les deux conteneurs vont, à travers une référence d'image ②, finalement créer et gérer une même ressource logique ①, qui sera seule responsable de la ressource physique ③. Le problème apparaît ici comme étant déplacé ailleurs plutôt que résolu. La résolution concrète vient du fait que la ressource mutualisé ① ne possède pas de paramètres, et il ne peut donc pas y avoir de conflit; si une ressource change le nom, cela va simplement crée une nouvelle ressource.

Dans l'exemple des conteneurs, la réconciliation se ferait donc comme suit:

1. La ressource représentant la configuration du conteneurs *A*, d'abord récupérer d'une manière ou d'une autre le nom complet de l'image configuré, avec son hash.
2. Avec cette information, la ressource va déclarer une dépendance sur une ressource mutualisée de type Image ayant pour nom le nom complet de l'image.
3. Cette ressource mutualisée de type image va s'occuper de *pull* l'image sur le disque. Cela sera fait en arrière plan car cela peut prendre plusieurs itérations. En attendant, la ressource aura le status pas prêt.
4. Tant que la ressource Image n'a pas le status terminé, la ressource Container ne fait rien d'autre.
5. Une fois l'image téléchargée, la ressource de type Image passe en status terminé.
6. Alors, la ressource de type Container va créer à proprement dit le conteneur.
7. *S'en suit le cycle de vie normal de la ressource.*
8. Lorsque l'une ou l'autre des ressources Container est supprimée, cela le lien de dépendance entre cette dernière et la ressource de type Image sera rompu.
9. Si le nombre de dépendance tombe à zéro sur la ressource de type Image, alors l'orchestrateur ordonne sa suppression, ce qui va effacer l'image du disque.
10. Dans le cas où d'autres conteneurs dépendraient sur Image, alors rien de plus ne se passerait.

### 2.1.5. Catégories de ressources

Les ressources sont classées en trois catégories selon leur rôle vis-à-vis des liens de possession et de dépendance: les ressources statiques, les ressources dynamiques, et les ressources mutualisées. Les ressources statiques sont créées et possédées par l'administrateur système (entité externe au modèle), elles constituent les racines de tout arbre de possession (l'administrateur système n'étant pas représenté comme une entité concrète). Elles ne peuvent être modifiées ou supprimées que par l'administrateur. Les ressources dynamiques sont toutes les ressources qui sont créées et possédées par une autre ressource, quelle que soit la catégorie de cette dernière. Le parent décide à lui seul quel est le contenu d'une telle ressource, en se basant ou non sur sa propre spécification, et quand en crée ou en supprime une. Enfin les ressources mutualisées sont créées de manière pseudo-implicite, lorsque une ressource *A*, peu importe sa catégorie, déclare une dépendance sur une ressource mutualisée *B*.

Le tableau 1 résume les propriétés et les liens pour chaque catégorie.

| Type             | Est créé par                | Est détenu par  | Lien de dépendance avec le créateur |
|------------------|-----------------------------|-----------------|-------------------------------------|
| <b>Statique</b>  | L'administrateur            |                 | Aucun                               |
| <b>Dynamique</b> | Une autre ressource         | Le créateur     |                                     |
| <b>Mutualisé</b> | Une ou plusieurs ressources | L'orchestrateur | Est une dépendance                  |

*Tableau 1. – Types de ressources et leur propriétés*

*Montre quel acteur une ressource peut-elle être créée, qu'elle lien aura-t-elle avec son créateur, et qui en sera le détenteur, autrement dit, qui peut la supprimer ou la modifier.*

Le tableau 1 synthétise les modes de création, de détention et les liens de dépendance propres à chaque catégorie de ressource: les ressources statiques, seules créées et détenues par l'administrateur, constituent les racines des arbres de possession sans lien de dépendance une quelconque autre ressource; les ressources dynamiques, créées par une autre ressource qui en devient le détenteur, sont liées à leur parent par un lien de possession et non par une dépendance; les ressources mutualisées, créées implicitement lorsqu'une ou plusieurs ressources déclarent une dépendance vers un même type et un même nom, sont détenues par l'orchestrateur et ne sont accessibles qu'à travers le lien de dépendance. Les ressources mutualisées sont elles aussi considérées comme des racine d'arbre, l'orchestrateur étant le système lui-même et non une entité spécifique au sein de celui-ci. Cette classification garantit qu'à tout instant chaque ressource possède un unique détenteur (administrateur, ressource parente ou orchestrateur).

En outre, l'introduction de ressources dynamiques et statiques permet de maintenir une stricte logique d'une ressource « virtuelle » = une ressource « physique ». Par exemple, il est commun dans les systèmes existant (netplan, systemd-networking, ...) de configurer le lien réseau, les routes et les adresses liées à ce lien au sein d'une seule et même « entité ». Cette logique peut être implémentée tel qu'illustré dans la figure 4.

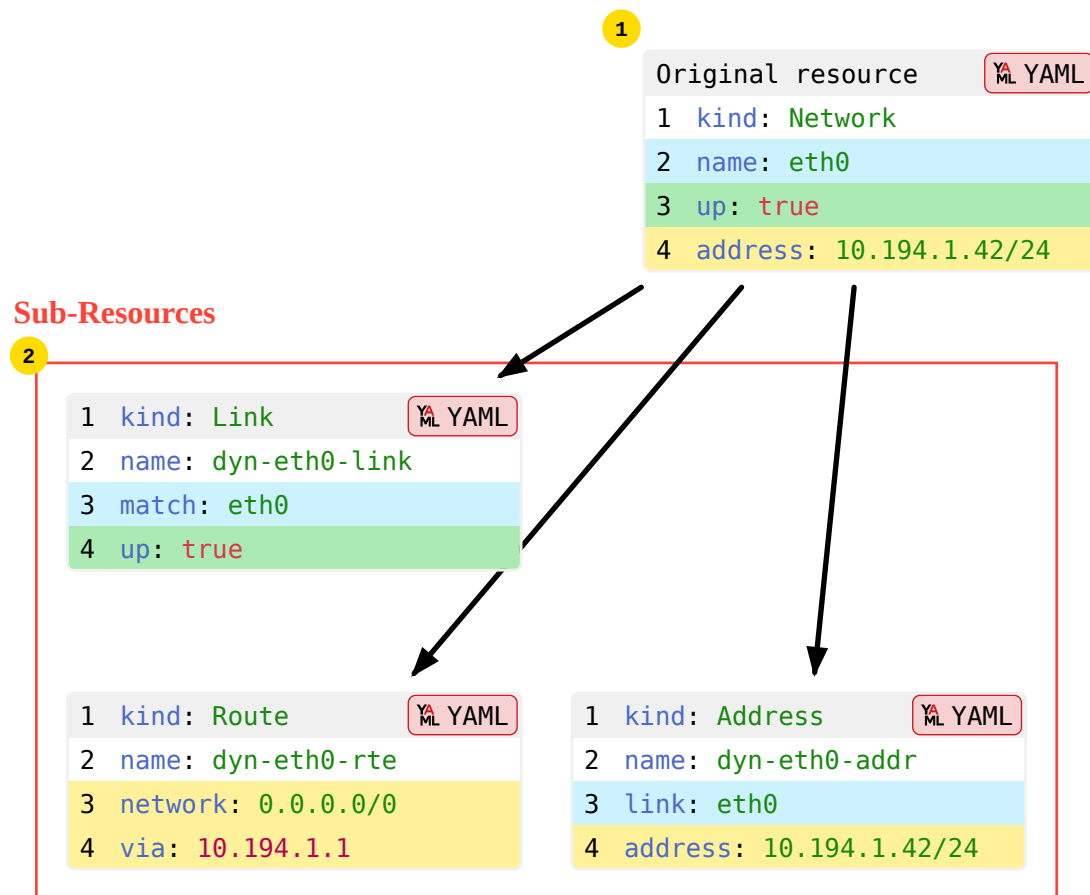


Figure 4. – Création de sous-ressources à partir d'un parent

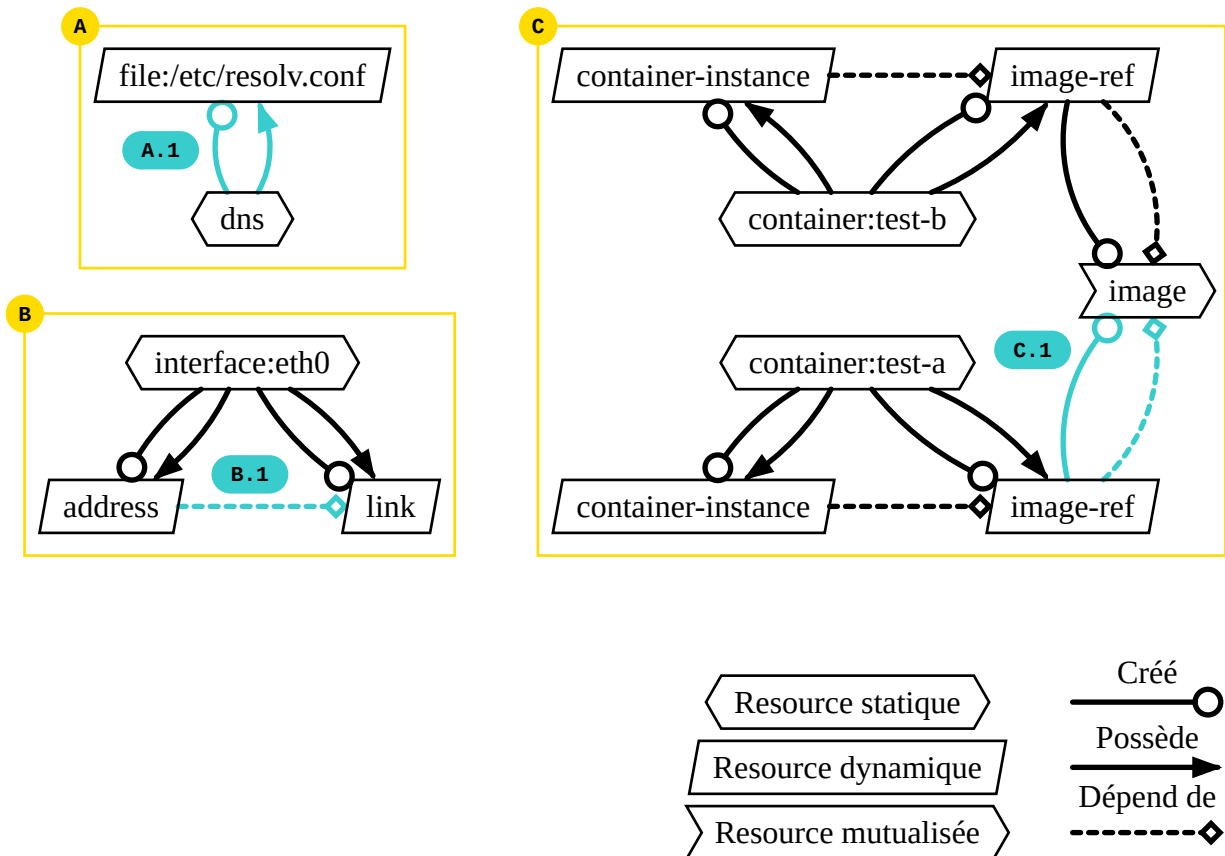
Illustre comment il est possible d'abstraire la gestion complexe de plusieurs ressources derrière une interface simple.

La [figure 4, Création de sous-ressources à partir d'un parent](#) illustre la manière dont une ressource parente de type Network **1** abstrait la complexité de la configuration réseau en créant et en possédant plusieurs sous-ressources dynamiques (Link, Address et Route) qui correspondent chacune à une entité physique distincte (lien réseau, adresse IP, route) **2**, conformément au principe « une ressource virtuelle = une ressource physique ». Le parent détermine librement la spécification de ces sous-ressources, en l'occurrence à partir de sa propre spécification déclarative (ici, les champs up et address sont propagés aux enfants avec les adaptations nécessaires).

### 2.1.6. Synthèse

#### TODO: Synthèse

- Introduire la synthèse
- Introduire la figure synthétique



## 2.2. Orchestrateur et réconciliation

### 2.2.1. Orchestrateur

L'orchestrateur est simplement le composant responsable de stocker l'ensemble des ressources du système, de s'assurer que les réconciliations ont lieu, et de garantir que le système se comporte correctement.

### 2.2.2. Contrôleur et déclarativité

La réconciliation est le processus qui assure que l'état observé d'une ressource converge vers sa spécification déclarative. À chaque cycle, le contrôleur lit l'état physique réel, le compare avec l'état désiré et produit les actions correctives nécessaires. La figure 6 schématise cette boucle générique.

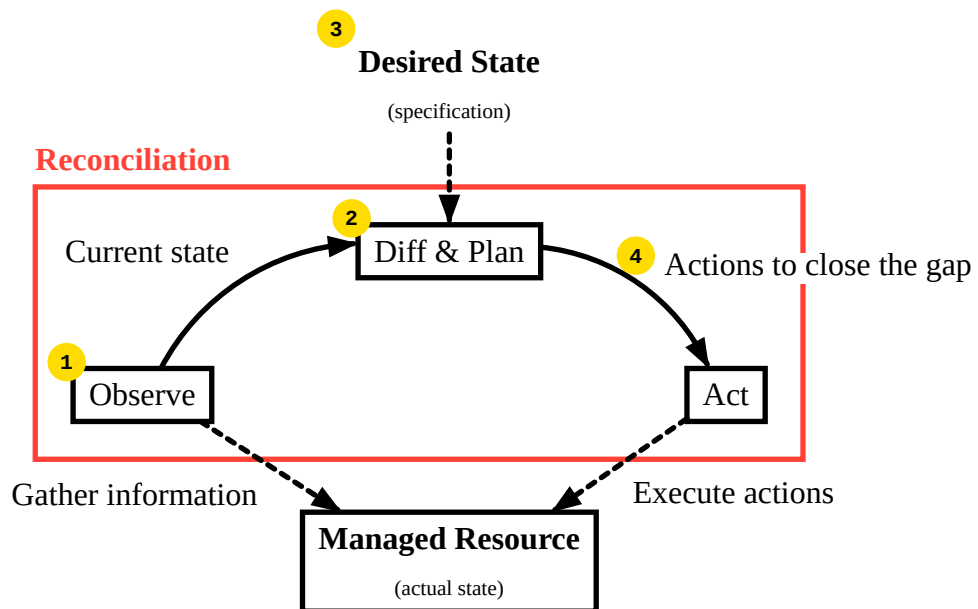


Figure 6. – Réconciliation d’une ressource

Le système observe l’état actuel de la ressource, calcule l’écart avec l’état désiré, et applique les actions correctives.

Par exemple, dans le cas de la configuration d’une interface réseau, il est spécifié que son status doit être « up », cela constitue donc l’état désiré 3. L’état actuel de l’interface (statut, adresse IP, etc.) est d’abord récupéré 1, puis comparé à l’état désiré 2. Dans le cas où le status serait « down », le système s’en rend compte et sait qu’il doit exécuter l’équivalent de `ip link set up` afin de mettre en route l’interface 4. Ce même mécanisme s’applique à toute modification ou suppression de la ressource: une modification de la configuration déclarative se traduit automatiquement par les actions correctives adéquates.

En ce qui concerne les sous-ressources, lors de la réconciliation d’une ressource donnée, celle-ci retourne d’une part le nouvel état observé, mais elle déclarera aussi l’ensemble de ses enfants avec leur spécification. Le ressource est libre de déterminer comme elle le souhaite le contenu de cette spécification, la seule contrainte étant la validité vis-à-vis du schéma du type de ressource. De fait, même en l’absence de tout paramètres, comme dans une ressource mutualisée, rien m’empêche, lors de la réconciliation, de créer des sous-ressources avec une spécification arbitraire. En outre, supprimé un enfant de cette déclaration revient à demander la suppression de celui-ci, processus qui est expliqué dans le [chapitre 2.2.5, Suppression des ressource](#).

Le contrôleur est le composant responsable d’un ou plusieurs types de ressources, pour lesquels il implémentera la logique déclarative: à partir de la spécification et de l’état observé d’une ressource, il détermine et exécute les actions correctives nécessaires pour faire converger l’état réel vers l’état désiré, puis retourne le nouvel état observé, le statut résultant, et la

liste des sous-ressources dont il déclare l'existence ou la dépendance. Le contrôleur ignore l'existence des autres ressources, même celles qu'il gère. Il maintient uniquement un état minimal lui permettant d'effectuer des tâches de longue durée (par exemple téléchargement). C'est uniquement lors de la réconciliation d'une ressource spécifique que le contrôleur prend connaissance de la spécification et des autres informations, de sorte à ce que la réconciliation d'une ressource puisse se baser uniquement sur les informations fournies par l'orchestrateur.

### 2.2.3. Orchestration de la réconciliation

Deux architectures d'orchestration sont envisageables.

Le système est composé de plusieurs ressources, chacune gérée par une boucle de logique de réconciliation indépendantes. Une réconciliation peut arriver de manière périodique ou en réponse à un événement, tant interne, tel qu'une mise à jour de la configuration, qu'externe, comme un conteneur qui s'arrête. À ce titre, deux modèles d'orchestration sont possibles: un modèle centralisé, qui contient une boucle unique, et au sein de cette boucle, fait appel à une méthode de réconciliation sur chaque ressource qui représente une itération. L'autre modèle est le modèle décentralisé, dans lequel chaque contrôleur contient sa propre boucle, organisé de la manière qui lui convient (une boucle par ressource, une boucle par type, etc.).

Le modèle centralisé est illustré comme suit:

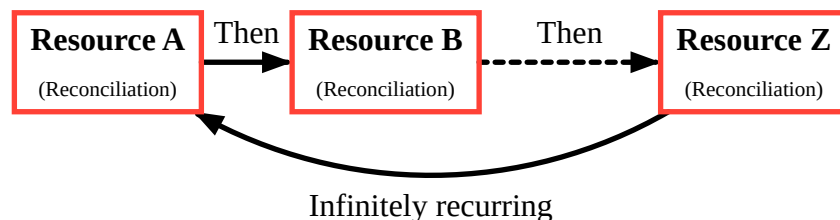


Figure 7. – Orchestration de la réconciliation dans un modèle centralisé

Illustre la manière donc la réconciliation individuelle de chaque ressource est intégrée vis-à-vis des autres ressources au sein d'un modèle de réconciliation centralisé.

<TODO> Commenter le diagram </TODO>

Tandis que le modèle décentralisé peut être illustré comme suit:

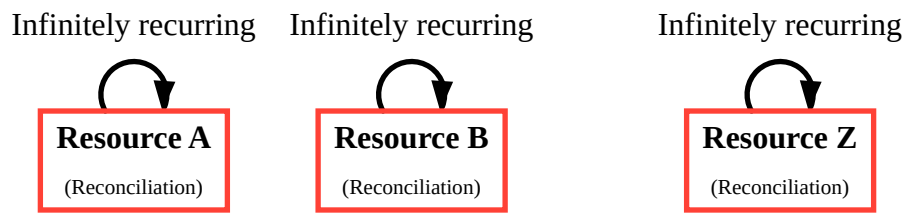


Figure 8. – *Orchestraion de la réconciliation dans un modèle décentralisé*  
*Illustre la manière donc la réconciliation individuelle de chaque ressource est intégrée vis-à-vis des autres ressources au sein d'un modèle de réconciliation décentralisé.*

<TODO> Commenter le diagram </TODO>

Le modèle centralisé dispose d'une vue globale de toutes les ressources et leur dépendance, ce qui permet de mieux coordonner la réconciliation à travers plusieurs contrôleurs. Dans le cadre de ce modèle, l'orchestrateur pourrait décider de ne pas réconcilier une ressource tant que ces dépendances ne sont pas prêtes. À l'inverse, dans le modèle décentralisé, la ressource serait quand même réconciliée quoi qu'il arrive. Toutefois le modèle décentralisé permet une plus grande flexibilité en termes de planification: chaque contrôleur peut déterminer lui-même l'intervalle d'exécution, etc. Cela peut toutefois être partiellement « mitigé » en ajoutant des paramètres contrôlant le timing au sein des réponses liées à la réconciliation.

En outre, le modèle centralisé permet d'aisément réagir aux changements d'état et de spécification des ressources. En effet, étant donné qu'un élément central coordonne le tout, il est trivial de propager les changements d'état ou re-déclencher les réconciliations dépendantes. Dans un modèle décentralisé, un mécanisme de notification entre les différents contrôleurs serait nécessaire et risque de complexifier le système. À l'inverse, le modèle décentralisé permet de réagir de manière beaucoup plus simple aux événements « externes » (par exemple, un conteneur qui s'arrête ou une interface réseau qui tombe). Pour supporter de genre de cas dans le modèle centralisé, il serait nécessaire d'ajouter une procédure permettant de notifier l'orchestrateur central.

En outre l'approche centralisée propose deux autres avantages mineurs: d'une part cela permet d'assigner trivialement le parent d'une ressource; en effet, étant donné que l'appel de réconciliation est effectué sur une ressource connue, et que sa réponse contient les sous-ressources a créé, il n'y a pas de risque qu'un parent soit mal assigné. En outre, l'orchestrateur centralisé permet d'aisément détecter lorsqu'un contrôleur est en panne: en effet, si celui-ci ne répond pas dans le temps imparti, on peut considérer qu'il est bloqué et prendre les actions appropriées.

Enfin, deux points supplémentaires existent, mais ne sont pas pris en compte: d'une part le modèle centralisé prend moins d'appels API (juste un), contrairement au système décentralisé qui a besoin d'au moins deux appels (l'un pour récupérer la config, l'autre pour push l'état). Cela n'est pas pris en compte, car il y a au final peu de ressources et les contrôleurs sont tous locaux, donc ce n'est pas important. Le modèle décentralisé permet une plus grande robustesse, car la panne est limitée à un nombre limité de ressources. Cela est toutefois sans importance, car les contrôleurs sont primordiaux et leur panne, mais en péril le système entier, donc tout s'arrête.

| Critère                                    | Pertinence | Faveur       | Justification                                                                                                                                                                                                                     |
|--------------------------------------------|------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Contrôle global de la planification</b> | Moyenne    | Centralisé   | Une boucle unique peut optimiser la réconciliation en tenant compte des dépendances et de la hiérarchie des ressources.                                                                                                           |
| <b>Flexibilité de planification</b>        | Moyenne    | Décentralisé | Chaque contrôleur peut régler son propre intervalle d'exécution, son backoff et sa concurrence sans coordination centrale.                                                                                                        |
| <b>Réaction aux événements internes</b>    | Moyenne    | Centralisé   | Déclencher une réconciliation sur changement d'état est trivial dans une boucle centrale; dans un modèle décentralisé, chaque contrôleur devrait gérer ses propres souscriptions.                                                 |
| <b>Réaction aux événements externes</b>    | Moyenne    | Décentralisé | Les événements externes (ex. arrêt d'un conteneur) peuvent être traités directement par le contrôleur concerné. Dans le modèle centralisé, ces signaux doivent transiter par l'orchestrateur.                                     |
| <b>Détection d'un contrôleur bloqué</b>    | Faible     | Centralisé   | L'orchestrateur central peut détecter un contrôleur bloqué via un timeout sur l'appel de réconciliation. Les contrôleurs décentralisés échouent silencieusement.                                                                  |
| <b>Assignation automatique du parent</b>   | Faible     | Centralisé   | La réponse de <code>reconcile()</code> contient les demandes de création, ce qui permet d'associer trivialement la relation parent-enfant.                                                                                        |
| <b>Nombre d'appels API</b>                 | Non retenu | Centralisé   | <code>reconcile()</code> regroupe l'état en un seul appel, contre plusieurs requêtes par itération en mode décentralisé. Non pertinent étant donné le faible nombre de ressources et l'absence de contrainte de débit.            |
| <b>Périmètre d'une panne</b>               | Non retenu | Décentralisé | Une panne de l'orchestrateur central arrête toutes les réconciliations. En mode décentralisé, la panne est isolée au contrôleur concerné. Non pertinent car la panne de n'importe quel contrôleur met en péril le système entier. |

Tableau 2. – Comparaison des modèles d'orchestration centralisés et décentralisés

Compare et synthétise les avantages et inconvénients de chaque modèle d'orchestration.

<TODO> Commenter le tableau </TODO>

Au regard du tableau 2, le modèle centralisé a été retenu comme modèle d'orchestration. Les avantages en termes de coordination des dépendances, de réaction aux événements internes et de détection de pannes l'emportent sur la flexibilité de planification et la capacité de réaction aux événements externes offerte par le modèle décentralisé. Les faiblesses du centralisé (réaction aux événements externes spontanés, flexibilité de la planification) sont compensées par un mécanisme explicite de notification: chaque contrôleur peut signaler un événement externe à l'orchestrateur, qui place alors la ressource concernée en tête de la file d'attente. Les aspects de robustesse et de charge d'appels API sont considérés comme secondaires dans un contexte où les contrôleurs sont locaux et en nombre limité.

#### 2.2.4. Boucle de réconciliation

L'orchestrateur maintient une file d'attente de ressources à réconcilier. Lors de l'insertion initiale d'une nouvelle ressource (= statut inconnu), ses dépendances sont parcourues en largeur (BFS) afin de garantir que les racines de l'arbre de dépendances soient réconciliées avant leurs feuilles. À chaque réconciliation, le contrôleur retourne une durée minimale à attendre avant la prochaine réconciliation de cette ressource, en l'absence d'événement. L'orchestrateur impose en outre un délai minimal global entre deux réconciliations pour éviter toute surcharge. Lorsque plusieurs ressources sont éligibles à la réconciliation (leur délai d'attente est écoulé), la ressource la plus en retard est traitée en premier. Une ressource peut également être réveillée prématurément par un événement interne (mise à jour d'une dépendance) ou externe (notification d'un contrôleur).

#### 2.2.5. Suppression des ressource

**TODO:** Suppression d'une ressource

- Que se passe-t-il quand une ressource est en cours de suppression et que une nouvelle dependance est ajoutée ou alors que la spec est modifiée? Rien: la dependance n'est pas « faite » et le système va remonter ça au contrôleur à la prochaine réconciliation.

Lorsqu'une ressource est supprimée, l'ensemble des ressources qu'elle possède est aussi supprimées. Afin de toujours garantir l'intégrité référentielle, la suppression part des feuilles (depth-first-search / DFS). En effet, une ressource ne peut pas être complètement retirée du système tant qu'elle possède d'autres ressources ou des dépendances entrantes. Dans le cas des dépendances entrantes, la suppression effective sera bloquée jusqu'à ce qu'il n'y ait plus aucune dépendance. Si une dépendance à une ressource en cours de suppression, ce lien de dépendance ne sera pas sauvegarder par le système. De même, si la spécification d'une ressource en cours de suppression est modifiée, cette modification sera elle aussi ignorée. Le système étant déclaratif,

si cette nouvelle version de la spécification persiste à être déclarée par une autre ressource, alors une fois la ressource entièrement supprimée, elle sera recrée.

En outre, comme indiqué dans le [chapitre 2.1.4, Ressources partagées et ressources mutualisées](#), les ressources mutualisées sont automatiquement placée en suppression dès lors qu'elles n'ont plus aucune dépendance entrante. La suppression de cette catégorie de ressource suit les mêmes règles que les autres ressources.

### 2.2.6. Réaction aux événements internes

**TODO:** Réaction aux événements internes

- Comment on gère la réconciliation lorsque l'orchestrateur reçoit une modification?
- => La ressource cible, après validation, est immédiatement ajoutée à la file de réconciliation
- **Question:** il faudrait sans doute permettre au contrôleur de retourner un délai minimal « fort » de sorte que  $\text{schedule\_time} = \max(\text{last} + \text{sys\_min}, \text{last} + \text{not\_before\_strong})$

La réconciliation est purement séquentielle. Il n'y a jamais plus d'une ressource de réconcilié en parallèle et de fait, il ne peut pas y avoir de problème de concurrence lors de la réconciliation. Afin de ne pas bloquer la boucle indéfiniment, une durée maximale par réconciliation est imposée. En outre, lorsque la réconciliation change d'état, en particulier lorsqu'une ressource passe d'un status quelconque à un status prêt ou terminé, les ressources qui dépendent dessus seront ajoutées à la file d'attente de réconciliation.

### 2.2.7. Réaction aux événements externes

**TODO:** Réaction aux événements externes

- Comment on gère la réconciliation lorsque par exemple un conteneur exit de manière inattendue?
- => Simplement permettre au contrôleur de demander à l'orchestrateur de placer une ressource (dont il est le contrôleur désigné) dans la file

Lorsque l'administrateur téléverse une nouvelle configuration, cela peut arriver à n'importe quel moment. Afin d'éviter tout problème, une ressource en cours de réconciliation est toujours verrouillée, ce qui mettra en suspend la modification de celle-ci jusqu'à ce que la réconciliation soit terminée. Pour éviter les problèmes, une durée maximale de réconciliation est imposée.

## 2.3. Initialisation du système

**TODO:** Initialisation du système

- Comment est-ce qu'un contrôleur est associé à une ressource? Durant la phase de démarrage, l'orchestrateur va faire un appel API particulier qui va retourner tous les types qu'un contrôleur sait gérer

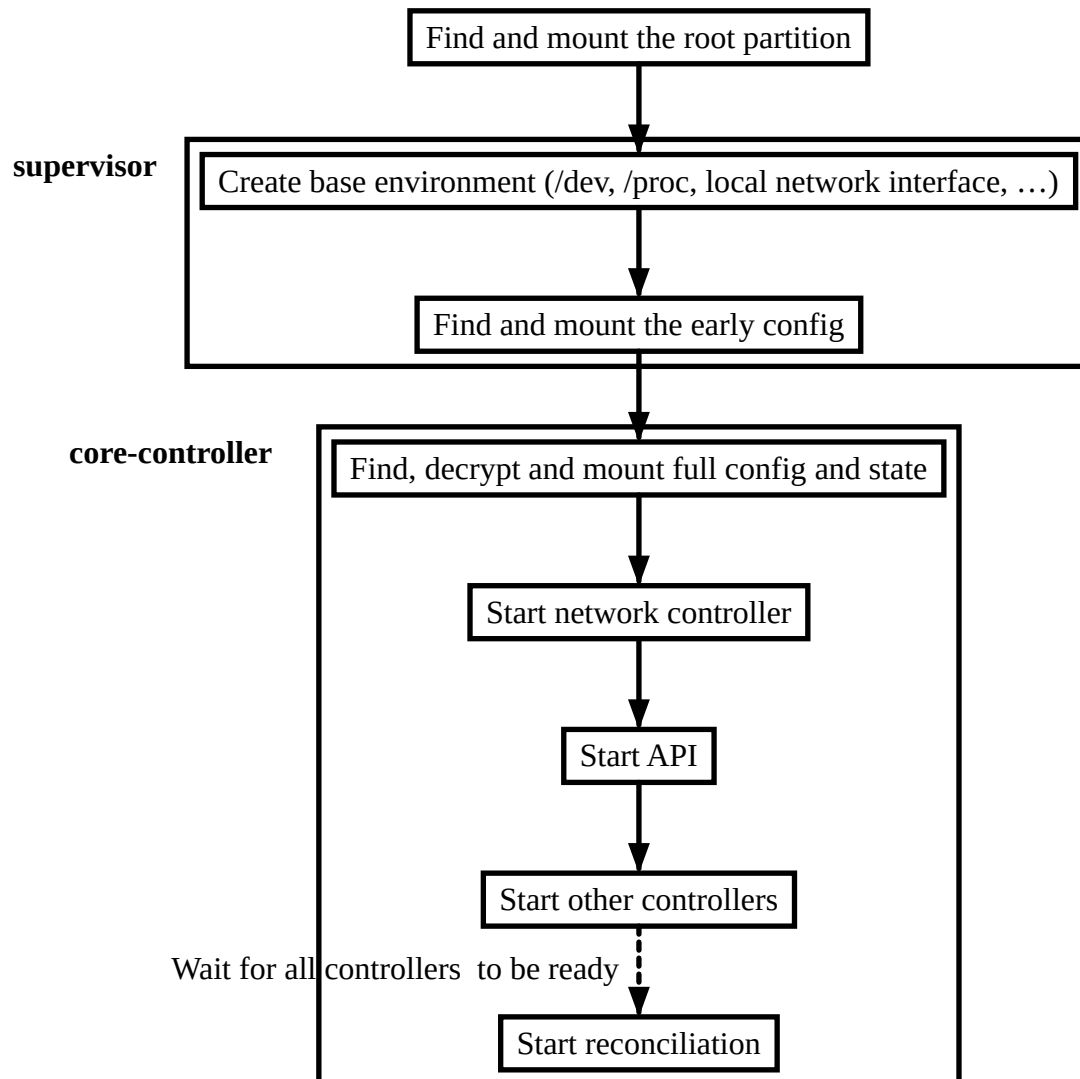


Figure 9. – Démarrage du système

Illustre les étapes principales du démarrage du système, avant que celui-ci n'entame la réconciliation.

## 2.4. Démarrage des processus

**TODO:** Démarrage des processus

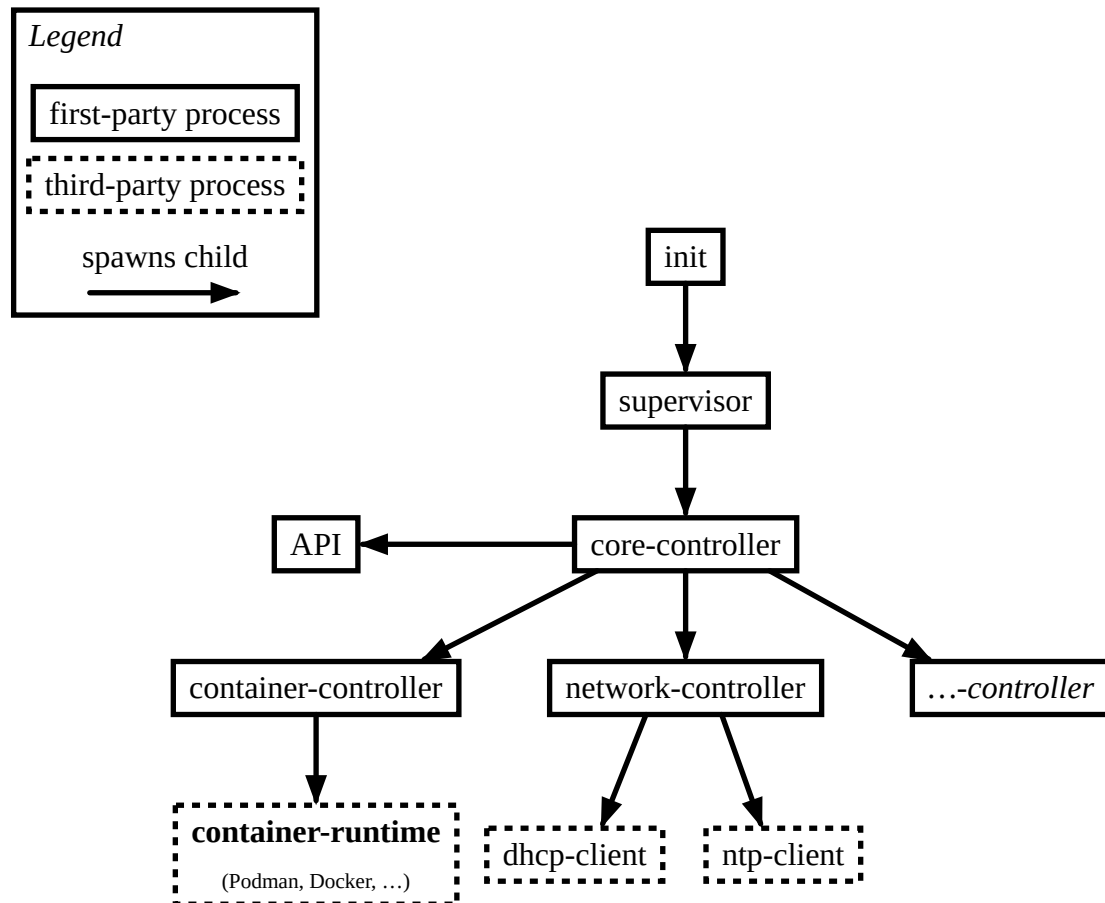


Figure 10. – Arbre d'exécution des processus du système

Illustre l'ordre dans lequel les processus du système sont démarrés, et qui les lance.

### 3. Implémentation

**Note:** Implémentation

- Parler de l'interface des interfaces des API
- Parler de l'interface pour la réconciliation
- Parler de l'environnement de build
- Parler de l'environnement de test (avec QEMU, les namespaces, etc)
- Coordination des ressources
  - Décrire la gestion de l'état désiré/courant
    - Scheduling (c.f. l'ADR 1)
  - gRPC
  - Décrire les API/services internes
- Parler de qualité de code (linting, CI/CD, etc)
- Parler de la sécurité

## 4. Tests et validation

**Note:** Tests et validation

- Scénarios de validation
- Benchmarking:
  - empreinte mémoire; pour ceci, deux méthodes:
    - pour qqc de précis, utiliser l’allocateur Peak en Rust
    - de manière plus générale donner X MB de RAM à la VM, et chercher la plus grosse allocation qu’on puisse faire depuis un conteneur
  - time-to-boot

## 5. Comparaison avec les solutions existantes

**Note:** Comparaison

Comparer à Talos Linux et NixOS par rapport aux benchmarks et aux scénarios.

## 6. Résultats

**Note:** Résultats

- ACADÉMIQUE + FONCTIONNEL
- Parler de la valeur ajoutée, des limitations, etc
- Dire que la solution a été validé sur l'ensemble des objectifs techniques énoncés
- Dire que la solution a été validé par rapport à d'autres solutions « concurrentes » avec satisfaction
- Dire que, outre les points énoncés, la solution remplit les objectifs personnels
- Dire que les objectifs du TB ont tous été atteints
- Parler de la valeur ajoutée d'un point de vue fonctionnel
- Parler (aussi!) des contraintes/points négatifs de la solution d'un point de vue fonctionnel

## 7. Discussion

**Note:** Discussion

- TECHNIQUE
- parler des bon et mauvais choix, limitations, etc
- Perspectives

## Conclusion